

Sardar Vallabhbhai National Institute of Technology (SVNIT), Surat
Department of Computer Science and Engineering
B.Tech II Semester-III
CS231 – Object Oriented Programming Practical Assignment
Lab Assignment - 10

1. Write a C++ program that takes a full name in the format:
 Firstname Middlename Surname
 Convert the entire string so that the first letter of each word is uppercase and the rest lowercase.
 Display the name in short form as Surname, F.M. (e.g., "Rahul Dev Sharma" → "Sharma, R.D.").
2. Write a menu-driven C++ program that does the following:
 - Ask the user to enter a sentence (single or multi-word string).
 - Count the frequency of each unique word (case-insensitive) in the sentence.
 - Display each word along with its frequency.
3. Write a C++ program that performs the following:
 - Accept a single string input from the user.
 - Remove all duplicate characters from the string while preserving the order of first occurrences.
 - Display the modified string.
4. Create a base class called Person with protected data members name (string) and age (int). Create a derived class Student that inherits publicly from Person. The Student class should have additional private data members: studentID (int) and major (string).
 - The Person class should have a constructor to initialize name and age, and a displayPerson() method to print these details.
 - The Student class should have a constructor that initializes all four data members
 - The Student class should have a displayStudent() method that first calls displayPerson() from the base class and then prints the studentID and major.
 - Write a main() function to create an object of the Student class, initialize it, and call its display method to show the complete details.
5. Create a base class Shape with a function displayArea().
Derive three classes:
 - Rectangle → inputs length and breadth
 - Triangle → inputs base and height
 - Circle → inputs radius

Each derived class should override displayArea() to show its own area.

6. Give the output of the below code and explain in a few lines the reason for the output you got.

a) `#include<iostream>`

`using namespace std;`

```
class BaseClass{
    int arr[10];
};
```

```
class DerivedBaseClass1: public BaseClass { };
```

```
class DerivedBaseClass2: public BaseClass { };
```

```
class DerivedClass: public DerivedBaseClass1, public DerivedBaseClass2{}
```

```
int main(void)
{
    cout<<sizeof(DerivedClass);
    return 0;
}
```

b) `#include<iostream>`

```
using namespace std;
class Scaler
```

```

{
    static int i;

    static
    {
        System.out.println("a");

        i = 100;
    }
}

public class StaticBlock
{
    static
    {
        System.out.println("b");
    }

    public static void main(String[] args)
    {
        System.out.println("c");

        System.out.println(Scaler.i);
    }
}

```

c) `#include<iostream>`

```

using namespace std;

class ClassA {
public:
    ClassA(int ii = 0) : i(ii) {}
    void show() { cout << "i = " << i << endl;}
private:
    int i;
};

class ClassB {
public:
    ClassB(int xx) : x(xx) {}
    operator ClassA() const { return ClassA(x); }
private:
    int x;
};

void g(ClassA a)
{ a.show(); }

int main() {
    ClassB b(10);
    g(b);
    g(20);
    getchar();
    return 0;
}

```

7. a) Explain the 'Diamond Problem' in Multiple Inheritance.

b) Give the code showing the 'Diamond Problem'.(Note : This code must give an error on executing.)

c) Explain how this 'Diamond Problem' can be solved.

8. Create two base classes:

- Teacher → with protected data members:
 - subject (string)
 - experience (int)
- Researcher → with protected data members:
 - researchArea (string)
 - publications (int)

Create a derived class Professor that inherits publicly from both Teacher and Researcher.

The Teacher and Researcher classes should each have a constructor to initialize their data members.

i). The Professor class should have an additional private data member:

name (string)

ii). The Professor constructor should initialize all data members from both base classes and its own.

iii). Create a displayProfessor() function that prints:

The professor's name

Teaching subject and experience

Research area and number of publications

iv) In main(), create an object of Professor, initialize it, and display all details.

9. **Write a C++ program** to demonstrate *Hybrid Inheritance* using the following structure:

- a. Create a base class **Person** with data members name and age.
- b. Derive a class **Student** from **Person**, adding studentID and course.
- c. Derive another class **Exam** from **Student**, adding marks.
- d. Create another class **Sports** (independent from Student) with a data member score.
- e. Finally, derive a class **Result** from both **Exam** and **Sports** to calculate and display the total performance. (*Note: The code should compile and demonstrate Hybrid Inheritance clearly.*)

10. Model a Smart IOT Device and develop a core firmware for centralized IoT platform that manages data from various smart devices (e.g. Watch, Camera, ...) uniformly, with each different type of device having unique functionality. Developing the system by demonstrating the inheritance concepts with the following requirements,

- a. Create a base class with common features and behaviours for smart devices. (e.g, *SmartIoTDevice* class with data member *deviceId*, *isConnected*, and member function *connect()* and *displayStatus()*)
- b. Create a derived class with unique features according to the type of device. (e.g. - *SmartWatch* class with *heartRate* and *CCTV* class with *isRecording* features)
 - i. *SmartWatch* - showcase connection through Wi-Fi
 - ii. *CCTV* - showcase connection through Bluetooth
 - iii. Add a behaviour (*processData()*) that processes the data relevant to the unique features added to the class and displays the status of each feature.
- c. Add one more smart device of your choice and implement its feature with appropriate behaviour. (similar to the *SmartWatch* and *CCTV*)
- d. Store the objects of all the different types of devices as a heterogeneous collection to manage them as a single unified type (base class). Iterate through each device from the collection and display the status of its features.

11. Demonstrate the Hierarchical and Multiple Inheritance concepts while implementing the Air Condition (AC) system using Sensor, Light and Controller components with the following requirements,

- a. Create a Base class (*Sensor*) with a data member (*reading*) and an appropriate member function to process the reading data.
- b. Create a Base class (*Controller*) with a data member (*autoMode*) and a member function to enable the auto mode.
- c. Create a derived class (*Light*) from the Base class (*SmartDevice*) with a data member (*brightnessLevel*) and an appropriate member function to adjust the brightness of the light.
- d. Derive *MotionSensor* from the *Sensor* class to detect motion. Read the data (through inherited *readData()* from base class) and update the status of motion detected or not.
- e. Derive *SmartVent* class from *Controller* and *Light* base classes with an additional feature (*airFlowRate*) and a member function to adjust the air flow rate.
- f. Demonstrate calls to the function inherited from the parent classes and display appropriate values. (Light - adjust brightness, Controller - adjust auto mode and SmartVent - air flow rate)
- g. Discuss whether the "Diamond Problem" issues persist in the given AC system or not, with appropriate reasons.

